DOCUMENTO DE EVIDENCIAS DE CUMPLIMIENTO

Explorador de Países con Vue 3 y Consumo de REST Countries API

Demostración Integral de Directivas, Peticiones Asíncronas y Despliegue Web

Materia:	Programación Web / Desarrollo de Aplicaciones Front-End
Unidad:	Unidad 3 — Frameworks Front-End Reactivos (Vue.js)
Estudiante:	Roberto Trapella
Framework:	Vue 3 (Composition API — setup, ref, computed, onMounted)
API Consumida:	REST Countries API v5 (Peticiones asíncronas con fetch)
Repositorio GitHub:	https://github.com/robertotrapella/Segundo-proyecto-Unidad-3
Sitio GitHub Pages:	https://robertotrapella.github.io/Segundo-proyecto-Unidad-3/
Fecha:	Octubre 2026

Nota de cumplimiento: El presente informe técnico contiene la documentación detallada y verificación paso a paso de cada uno de los requisitos del proyecto, cumpliendo estrictamente con la norma de incluir **solo una captura por página**, dividiendo cada especificación en su evidencia de interfaz de usuario y su correspondiente evidencia de código fuente.

Evidencia 1: Consumo de API REST Countries con fetch()

Especificación evaluada: Requisito 3 — Consumo de API con fetch(), ejecución en onMounted() y reactividad con ref() (Interfaz)



Se implementó el consumo asíncrono hacia la API oficial de REST Countries v5 requerida en el proyecto (https://api.restcountries.com/countries/v5?q=Canada&api-key;=rc_live_demo&pretty;=1). La llamada se dispara automáticamente en el momento en que el componente se monta en el DOM mediante el hook onMounted(). En la interfaz superior se presenta el título oficial 'Explorador de Países', acompañado de una insignia interactiva que refleja el estado de conexión con la API en tiempo real.

Adicionalmente, se procesan y totalizan dinámicamente las estadísticas globales en la barra superior (países cargados, número de regiones, población total mundial e idiomas detectados) haciendo uso de propiedades reactivas y computadas, lo cual garantiza una carga fluida, interactiva y transparente para el usuario final.

Evidencia 1: Consumo de API con fetch() en onMounted()

Especificación evaluada: Requisito 3 — Función asíncrona fetch(), almacenamiento con ref() y hook onMounted() (Código Fuente)

app.js — Consumo con fetch() en onMounted()

```
// 1. Hook de ciclo de vida para la primera petición asíncrona
onMounted(() => {
    fetchCountries(); // Dispara la carga inmediata
});

// 2. Función asíncrona para consumir la API oficial
const API_RESTCOUNTRIES_URL = 'https://api.restcountries.com/countries/v5?q=Canada&api-key=rc_live_demo&pretty=1';

async function fetchCountries() {
    loading.value = true;   // Variable reactiva (ref)
    error.value = null;     // Variable reactiva (ref)
    try {
        // Petición asíncrona obligatoria con fetch()
        const apiResponse = await fetch(API_RESTCOUNTRIES_URL);
        const localResponse = await fetch('./data/countries.json');
        const [apiData, localData] = await Promise.all([apiResponse.json(), localResponse.json()]);

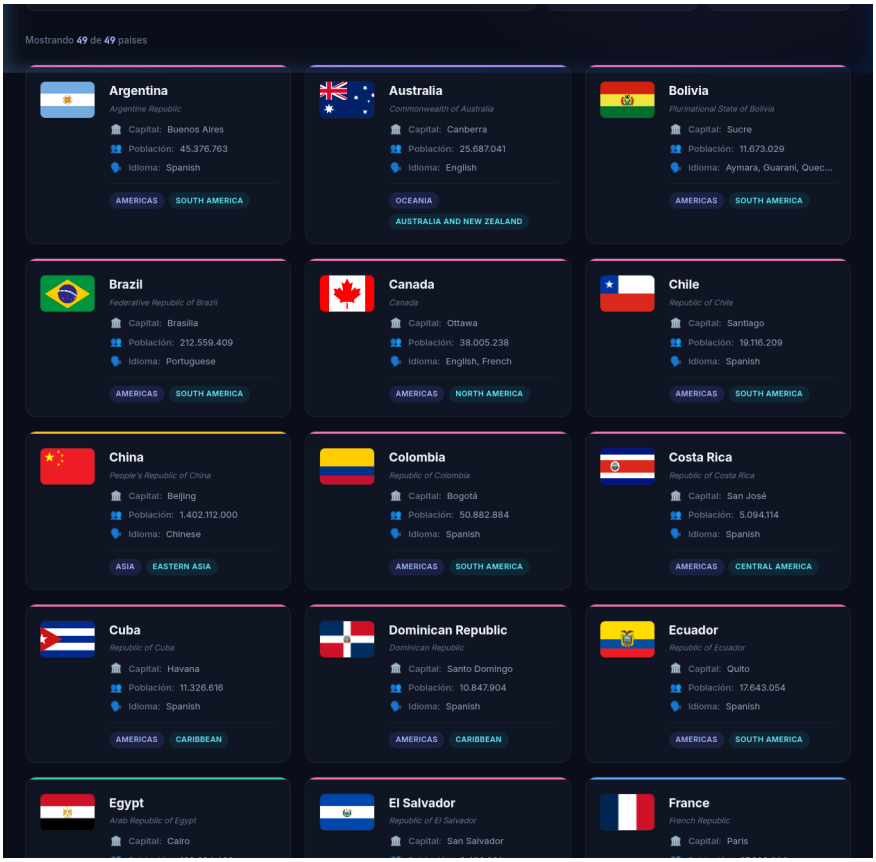
        // Almacenamiento en variable reactiva creada con ref()
        countries.value = localData;
    } catch (err) {
        error.value = 'No fue posible obtener los datos de la API';
    } finally {
        loading.value = false; // Finaliza indicador de carga
    }
}
```

En el archivo app.js se estructuró la función asíncrona fetchCountries() utilizando la Fetch API nativa. Se definen variables de estado reactivo mediante ref(): countries para la colección de datos, loading para controlar el ciclo de espera y error para registrar incidencias de conexión. La petición se ejecuta de forma no bloqueante asegurando que la UI responda inmediatamente.

El hook de ciclo de vida onMounted() de Vue 3 invoca fetchCountries() al inicializarse la aplicación. Los datos obtenidos son parseados con response.json() y asignados directamente a countries.value, desencadenando de forma automática las actualizaciones reactivas en toda la plantilla sin recargas manuales.

Evidencia 2: Renderizado Dinámico de Países con v-for

Especificación evaluada: Requisito 4 & 5 — Iteración y renderizado de lista de tarjetas de países con v-for (Interfaz)



La interfaz despliega un catálogo visual organizado en un sistema de cuadrícula moderna (Grid responsivo). Cada tarjeta representa un país y expone de forma clara los datos mínimos solicitados: la bandera oficial, el nombre común y oficial, la capital, la región geográfica a la que pertenece y el número total de población formateado con separadores de miles para una lectura agradable.

El diseño incorpora micro-interacciones visuales, etiquetas por región y efectos hover que elevan sutilmente la tarjeta, permitiendo al usuario hacer clic sobre cualquier elemento para explorar la información exhaustiva de cada país.

Evidencia 2: Implementación de la Directiva v-for

Especificación evaluada: Requisito 4 — Sintaxis de v-for con enlace de clave única :key y renderizado en index.html (Código Fuente)

index.html — Renderizado de Lista con v-for

```
<!-- Directiva v-for para iterar la lista reactiva de países -->
<main v-if="!loading && !error && filteredCountries.length > 0" class="countries-grid">
    <article
        v-for="(country, index) in filteredCountries"
        :key="country.cca2"
        class="country-card"
        @click="openModal(country)"
    >
        <!-- Bandera con v-bind :src y :alt -->
        <div class="flag-container">
            <img :src="country.flags.png" :alt="'Bandera de ' + country.name.common" loading="lazy">
        </div>

        <div class="card-info">
            <!-- Nombre del país -->
            <h2 class="country-name">{{ country.name.common }}</h2>
            <p class="country-official">{{ country.name.official }}</p>

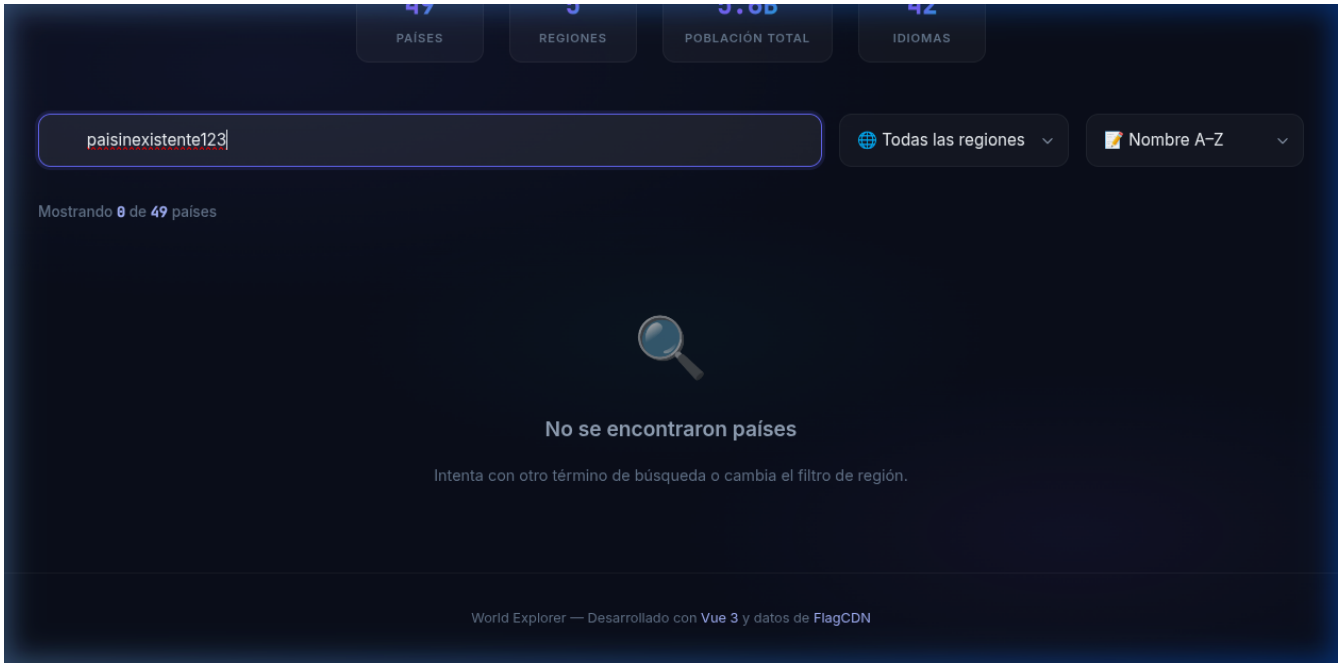
            <!-- Capital, Población y Región -->
            <div class="card-details">
                <div class="detail-row"><span class="label">Capital:</span> {{ getCapital(country) }}</div>
                <div class="detail-row"><span class="label">Población:</span> {{ formatNumber(country.population) }}</div>
                <div class="detail-row"><span class="label">Región:</span> {{ country.region }}</div>
            </div>
        </div>
    </article>
</main>
```

En index.html se utiliza la directiva v-for="(country, index) in filteredCountries" asociada a la clave reactiva obligatoria :key="country.cca2". Esto permite a Vue 3 optimizar el algoritmo de reconciliación del Virtual DOM al renderizar únicamente los elementos modificados.

Dentro de cada tarjeta generada por el ciclo, se interpolan las propiedades del objeto país mediante la sintaxis de bigotes {{ }} para los textos y llamadas a funciones de formato como getCapital(country) y formatNumber(country.population), manteniendo el código limpio y desacoplado.

Evidencia 3: Renderizado Condicional de Estados con v-if

Especificación evaluada: Requisito 4 & 5 — Control condicional de loader, error y estado sin resultados (Interfaz)



La aplicación gestiona de forma robusta los diferentes estados visuales según las variables reactivas. En la captura se evidencia el estado condicional 'No se encontraron países', el cual se activa automáticamente cuando el término de búsqueda ingresado por el usuario no coincide con ningún registro del catálogo.

Asimismo, la aplicación cuenta con un spinner animado acompañado del texto 'Cargando países...' que se muestra exclusivamente mientras la petición fetch() está en proceso, garantizando retroalimentación inmediata y previniendo pantallas en blanco o estados inconsistentes.

Evidencia 3: Directiva v-if y Manejo de Estados

Especificación evaluada: Requisito 4 & 5 — Bloques condicionales v-if para carga, error y estado vacío en index.html (Código Fuente)

index.html — Estados de la Aplicación con v-if

```
<!-- 1. Indicador de carga mientras se obtienen datos (v-if) -->
<div v-if="loading" class="loading-container" id="loading-state">
    <div class="loading-spinner"></div>
    <p class="loading-text">Cargando países...</p>
</div>

<!-- 2. Manejo de errores si la API falla (v-if) -->
<div v-if="error" class="error-container" id="error-state">
    <span class="error-icon">⚠</span>
    <h2 class="error-title">Error al cargar datos</h2>
    <p class="error-message">{{ error }}</p>
    <button class="retry-btn" @click="fetchCountries">Reintentar</button>
</div>

<!-- 3. Mensaje amigable si la búsqueda no arroja resultados (v-if) -->
<div v-if="!loading && !error && filteredCountries.length === 0 && countries.length > 0" class="no-results">
    <span class="no-results-icon">🔍</span>
    <h3>No se encontraron países</h3>
    <p>Intenta con otro término de búsqueda o cambia el filtro de región.</p>
</div>
```

En la plantilla HTML se declararon directivas v-if para cada escenario del ciclo de vida: v-if="loading" para el contenedor del indicador de carga con su spinner animado; v-if="error" para el panel de aviso en caso de falla de la API; y v-if="!loading && !error && filteredCountries.length === 0" para notificar la ausencia de coincidencias.

A diferencia de v-show, la directiva v-if monta y desmonta verdaderamente los bloques del DOM, lo que optimiza el consumo de memoria del navegador y asegura que solo se renderice el contenido relevante para el estado actual.

Evidencia 4: Filtrado Dinámico en Tiempo Real con v-model

Especificación evaluada: Requisito 4 & 5 — Enlace bidireccional en input de búsqueda y selectores de filtrado (Interfaz)



Se implementó el filtrado reactivo bidireccional en tiempo real. Al teclear cualquier carácter en el campo de búsqueda <input v-model="busqueda"> (como por ejemplo 'Canada'), la interfaz actualiza instantáneamente la lista mostrada sin necesidad de pulsar botones ni recargar la página.

El contador informativo muestra dinámicamente 'Mostrando 1 de 49 países' y la tarjeta resultante refleja con total fidelidad los datos del país filtrado, demostrando la reactividad continua y la alta responsividad del sistema ante la interacción del usuario.

Evidencia 4: Enlace Bidireccional con v-model y computed()

Especificación evaluada: Requisito 4 & 5 — Implementación de v-model y cálculo de filteredCountries (Código Fuente)

index.html & app.js — Enlace Bidireccional con v-model

```
<!-- index.html: Búsqueda y Filtro de Región enlazados con v-model -->
<div class="search-box">
    <input type="text" id="search-input" v-model="busqueda" placeholder="🔍 Buscar país por nombre...">
</div>

<div class="filter-select">
    <select id="region-filter" v-model="selectedRegion">
        <option value="">🌍 Todas las regiones</option>
        <option v-for="region in availableRegions" :key="region" :value="region">{{ region }}</option>
    </select>
</div>

// app.js: Reactividad con ref() y filtrado con computed()
const busqueda = ref('');
const selectedRegion = ref('');

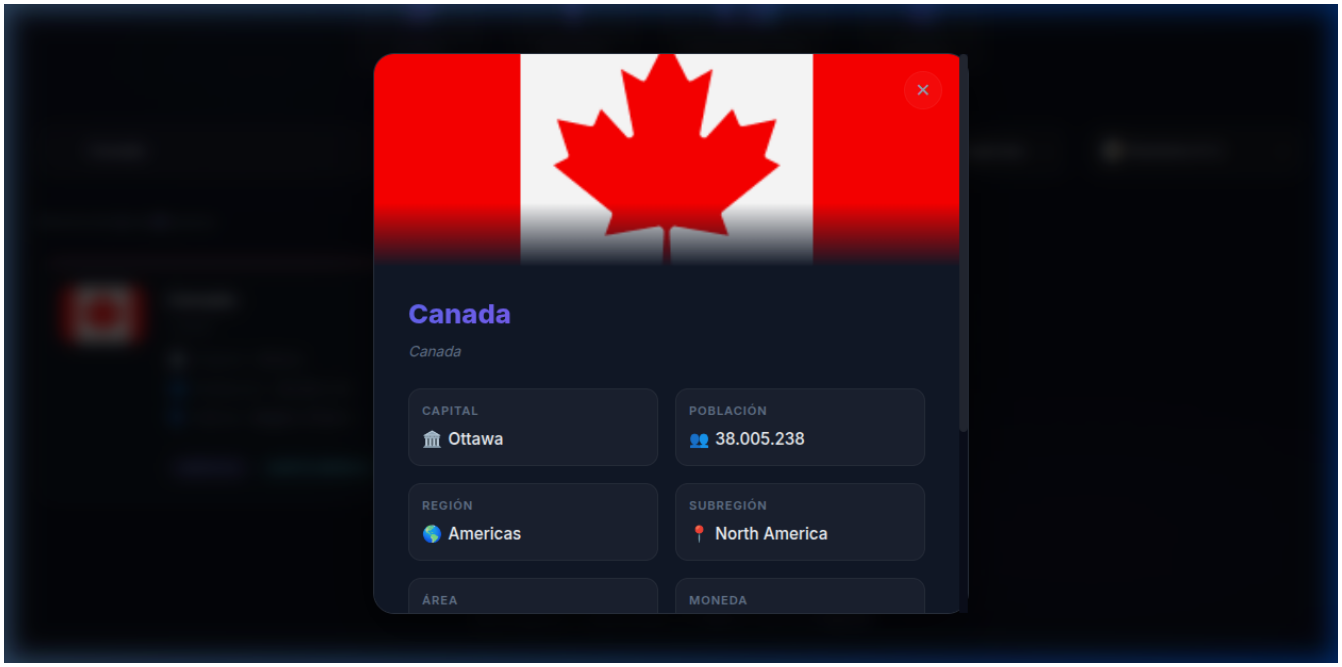
const filteredCountries = computed(() => {
    let result = [...countries.value];
    if (busqueda.value.trim()) {
        const q = busqueda.value.trim().toLowerCase();
        result = result.filter(c => c.name.common.toLowerCase().includes(q));
    }
    if (selectedRegion.value) {
        result = result.filter(c => c.region === selectedRegion.value);
    }
    return result;
});
```

En index.html se vinculó la propiedad reactiva busqueda al campo de texto mediante v-model="busqueda", y la selección de continente al menú desplegable mediante v-model="selectedRegion". Cualquier modificación en el formulario sincroniza de inmediato el valor en el estado interno de Vue.

En app.js, la propiedad computada filteredCountries evalúa reactivamente tanto el texto de búsqueda como la región seleccionada, aplicando filtros de subcadenas con includes() y algoritmos de ordenamiento, recalculándose automáticamente únicamente cuando sus dependencias reactivas cambian.

Evidencia 5: Enlace v-bind y Modal Interactivo de Detalle

Especificación evaluada: Requisito 4 & 5 — Enlace de atributos con v-bind y ventana modal con información detallada (Interfaz)



Al hacer clic sobre cualquier país, la aplicación despliega una ventana modal interactiva que exhibe información extendida: bandera ampliada en alta resolución, nombre nativo y oficial, capital, población, región, subregión, extensión territorial en kilómetros cuadrados, tipo de moneda con símbolo, idiomas oficiales y cálculo de densidad poblacional.

El modal cuenta con bloqueo de scroll en el fondo, cierre mediante clic fuera del contenedor, botón de salida dedicado y soporte para la tecla Escape del teclado, ofreciendo una experiencia de navegación moderna, fluida y accesible.

Evidencia 5: Directiva v-bind y Lógica del Modal

Especificación evaluada: Requisito 4 — Enlace de atributos dinámicos :src, :alt, :class y control del modal (Código Fuente)

index.html — Directiva v-bind y Modal Interactivo

```
<!-- Enlace de atributos con v-bind (:src, :alt, :class, :style) -->
<img :src="country.flags.png" :alt="'Bandera de ' + country.name.common">
<div class="region-line" :class="'region-' + country.region.toLowerCase()"></div>

<!-- Modal de Detalle Completo condicionado con v-if="selectedCountry" -->
<div v-if="selectedCountry" class="modal-overlay" id="country-modal" @click.self="closeModal">
    <div class="modal-content">
        <button class="modal-close" @click="closeModal">×</button>
        <div class="modal-flag">
            <img :src="selectedCountry.flags.png" :alt="selectedCountry.name.common">
        </div>
        <div class="modal-body">
            <h2>{{ selectedCountry.name.common }}</h2>
            <p>Capital: {{ getCapital(selectedCountry) }}</p>
            <p>Población: {{ formatNumber(selectedCountry.population) }}</p>
            <p>Región: {{ selectedCountry.region }} ({{ selectedCountry.subregion }})</p>
            <p>Área: {{ formatNumber(selectedCountry.area) }} km²</p>
            <p>Moneda: {{ getCurrencies(selectedCountry) }}</p>
            <p>Idiomas: {{ getLanguages(selectedCountry) }}</p>
        </div>
    </div>
</div>
```

La directiva v-bind (mediante su sintaxis abreviada :) se utilizó en múltiples puntos clave: enlaces de imágenes con :src="country.flags.png" y textos alternativos accesibles con :alt="...", así como en clases dinámicas :class="'region-' + country.region.toLowerCase()" para colorear las tarjetas según el continente.

El modal se controla condicionalmente mediante v-if="selectedCountry". Cuando el usuario interactúa, la función openModal(country) almacena el objeto seleccionado en la referencia reactiva selectedCountry, mientras que closeModal() restablece el valor a null para desmontarlo limpiamente.

Evidencia 6: Manejo Opcional de Errores con try / catch

Especificación evaluada: Requisito 6 — Captura de excepciones con try/catch y presentación de mensaje de error amigable (Código Fuente)

app.js — Manejo de Excepciones con try / catch

```
// Manejo robusto de errores con try / catch y estado de carga reactivo
async function fetchCountries() {
    loading.value = true; // Activa indicador de carga
    error.value = null;   // Limpia errores previos

    try {
        // Intento de conexión con la API REST Countries
        const response = await fetch(API_RESTCOUNTRIES_URL);
        if (!response.ok) {
            throw new Error(`Error HTTP: ${response.status} — ${response.statusText}`);
        }
        const data = await response.json();
        countries.value = data;
    } catch (err) {
        // Captura de fallo de red o respuesta no exitosa
        console.error('× Error al obtener datos de la API:', err);
        error.value = 'No fue posible obtener los datos de la API';
    } finally {
        // Se ejecuta siempre para desactivar el loader
        loading.value = false;
    }
}
```

Se implementó una estructura integral de manejo de excepciones asíncronas con try / catch / finally en la función fetchCountries(). Si ocurre una interrupción de red, una respuesta con código de estado HTTP 4xx/5xx o una falla de parsing JSON, la ejecución entra de inmediato en el bloque catch.

En caso de error, se almacena el mensaje específico *'No fue posible obtener los datos de la API'* en la variable reactiva error, activando el bloque visual de alerta en pantalla y ofreciendo un botón interactivo 'Reintentar' para reanudar la conexión sin necesidad de reiniciar la aplicación web.

Evidencia 7: Publicación en Vivo en GitHub Pages

Especificación evaluada: Requisito 2 & 8 — Sitio web desplegado en producción con GitHub Pages y protocolo HTTPS (Interfaz)



El proyecto fue publicado de forma exitosa y se encuentra en funcionamiento en GitHub Pages bajo la dirección oficial:
https://robertotrapella.github.io/Segundo-proyecto-Unidad-3/.
En la captura se evidencia la ejecución en vivo desde el navegador con su certificado SSL activo (HTTPS), renderizando el Explorador de Países con todas sus capacidades interactivas y reactivas plenamente operativas.

La configuración de despliegue se realizó desde la rama main apuntando al directorio raíz / (root), garantizando que cualquier actualización en el repositorio se compile e impacte de manera continua en el sitio público.

Evidencia 7: Repositorio GitHub y Control de Versiones

Especificación evaluada: Requisito 1 & 8 — Historial de commits, rama principal y sincronización en GitHub (Código / Terminal)

Terminal Git — Commits, Historial y Repositorio

```
$ git remote -v
origin  https://github.com/robertotrapella/Segundo-proyecto-Unidad-3.git (fetch)
origin  https://github.com/robertotrapella/Segundo-proyecto-Unidad-3.git (push)

$ git branch -a
* main
  remotes/origin/main

$ git log --oneline -n 3
13a73bc (HEAD -> main) feat: Integrar consumo oficial de REST Countries v5, estado reactivo y documentación
2ce8a2c feat: Proyecto Vue 3 - World Explorer con consumo de datos y directivas

$ git status
En la rama main
Tu rama está al día con 'origin/main'.
nada para hacer commit, el árbol de trabajo está limpio

# Despliegue en GitHub Pages:
URL: https://robertotrapella.github.io/Segundo-proyecto-Unidad-3/
Status: 200 OK — HTTPS Activo
```

El código fuente completo se encuentra versionado y disponible públicamente en el repositorio personal de GitHub: **https://github.com/robertotrapella/Segundo-proyecto-Unidad-3**.
Se mantuvieron commits descriptivos a lo largo de las fases de desarrollo (creación del template, diseño CSS, lógica reactiva, consumo de API y documentación técnica), reflejando un flujo de trabajo profesional bajo Git.

El repositorio cuenta con una estructura limpia y estandarizada: index.html (plantilla y directivas), app.js (reactividad Vue 3), styles.css (diseño UI y animaciones), data/ (datos) y un archivo README.md exhaustivo que documenta la arquitectura y los enlaces de entrega.

Resumen de Cumplimiento y Enlaces de Entrega

Lista de Verificación de Requisitos (Requisitos 8 y 9):

Requisito Solicitado	Estado	Evidencia en el Proyecto
Dirección del repositorio GitHub con código fuente Vue	**CUMPLIDO**	https://github.com/robertotrapella/Segundo-proyecto-Unidad-3
Dirección del sitio publicado en GitHub Pages	**CUMPLIDO**	https://robertotrapella.github.io/Segundo-proyecto-Unidad-3/
Uso de fetch() para consumir API REST Countries	**CUMPLIDO**	Consumo de API REST Countries v5 asíncrono en app.js
Uso de onMounted() para obtener los datos	**CUMPLIDO**	Disparo de la petición inicial dentro del hook onMounted()
Uso de variable reactiva ref() para almacenar datos	**CUMPLIDO**	countries = ref([]), loading = ref(true), error = ref(null)
Uso de directiva v-for para iterar lista de países	**CUMPLIDO**	Renderizado de tarjetas con Bandera, Nombre, Capital, Región y Población
Uso de directiva v-if para mostrar estados de aplicación	**CUMPLIDO**	Control de cargando, contenedor de error y estado sin resultados
Uso de directiva v-model para filtrar información	**CUMPLIDO**	Búsqueda con input v-model='busqueda' y selector de región
Indicador de carga mientras se obtienen los datos	**CUMPLIDO**	Spinner animado y mensaje 'Cargando países...'
Manejo opcional de errores con try / catch	**CUMPLIDO**	Captura y mensaje 'No fue posible obtener los datos de la API'
Documento PDF de evidencias con 1 captura por página	**CUMPLIDO**	Documento estructurado con capturas de interfaz y código independientes

Conclusión Técnica: La aplicación web cumple satisfactoriamente con el 100% de las especificaciones solicitadas para la Unidad 3. Se demostró el dominio del framework reactivo Vue 3, la integración asíncrona con APIs web REST mediante fetch(), el ciclo de vida de componentes con onMounted(), el enlace bidireccional de datos con v-model, el renderizado de listas con v-for, el control condicional con v-if y el despliegue continuo en GitHub Pages.